

Large Language Models and Alignment

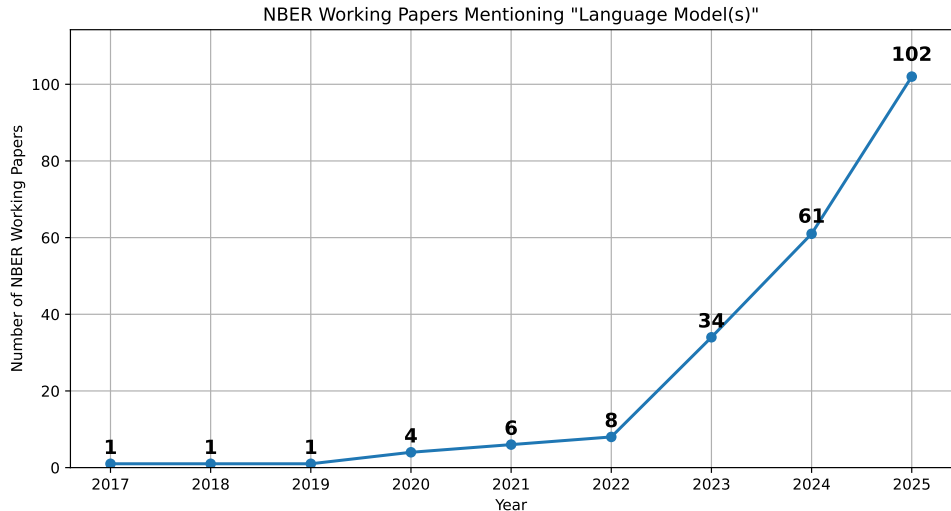
Summer School in AI & Applied Economics

Elliott Ash

ETH Zürich

August 2026

NBER Working Papers Mentioning “Language Model(s)”



What are language models?

- ▶ **Language modeling** is the task of teaching an algorithm to predict and generate language.
- ▶ The earliest computational language model traces to Shannon (1948) — followed by very slow progress for 75 years.
- ▶ In the mid/late 2010s, sudden rapid progress, brought on by:
 - ▶ **data**: massive, accessible collections of text corpora (the internet);
 - ▶ **compute**: GPU processors specialized for large-scale dense matrix multiplications.

LLM = “Large Language Model”

An algorithm that predicts a word in the middle or at the end of a sequence (following a **model** of **language**), using **large**-scale compute to process **large** databases of text.

What are *aligned* language models?

- ▶ LLMs trained only on word prediction are now called **base models**.
 - ▶ Base LLMs generate the *most likely* sequence, based on the training corpus.
- ▶ The LLMs underlying AI assistants since ChatGPT are called **aligned models** (or **instruct models**).
 - ▶ Aligned LLMs generate the sequence **most likely to benefit the user**.

Today's lecture, in one line

How we get from the base model to the aligned model — and what each step means for using LLMs in research.

Why do LLMs matter in economics?

1. An important **technology** that will have economic impacts.
2. For **measuring** the content of unstructured text documents.
 - ▶ (mostly) comparable to having well-trained humans read and code documents, at high speed and low cost.
3. For **facilitating the research process**.
 - ▶ writing, coding, literature review;
 - ▶ qualitative interviews and data collection.

Easy Interviews

Elliott Ash
ETH Zurich

Soumitra Shukla
Harvard Business School

Jason Sockin
Cornell University

AEAs 2026

Report on a recent job interview



Please adhere to the Code of Conduct



We review and approve each post based on our [Code of Conduct](#) before it is published on the website.

Employer *

IZA

Rate the experience overall*



Positive

Job title *

Postdoctoral Scholar

Describe the interview process*

test test test test test test test test test test
test test test test test test test test test test
test test test test test test test test test test

At least 30 words

Difficulty level of interview *

Difficult



Have you received an offer? *

Yes, and I accepted it



Interview questions*

Ask*

Would you move to Germany?



How did you answer this question? [Optional]



Add question

Further

LLM Prompts for Colleague Ability and Accepting Anyone

I will provide you with a text snippet that describes a job interview.

Colleague ability. Does the description say that prospective colleagues are highly able, intelligent, or competent? Determine whether the interview description explicitly mentions or strongly implies that colleagues at the employer are of high ability, intelligence, or competence.

"They'd accept anybody". Does the description suggest, through explicit mention or clear implication, that the employer would hire anyone? That is, implying that the employer is desperate for employees and would accept any applicant regardless of qualifications or fit.

Read the description and provide a label "Definitely Yes", "Probably Yes", "Probably Not", "Definitely Not", or "N/A", followed by a semicolon (;) and then a short one-sentence explanation.

Applying a Large Language Model (LLM) to Text

Ask Gpt-4o-mini to annotate a sample of interview processes

- 500 interviews for each industry x difficulty level
- Using these as a training set, scale up to all interviews using RoBERTa model

Then do human inspection and qualitative evaluation to validate output from LLM.

- LLM annotations seem to compare well to human judgment
 - Ask 2 humans to annotate a sample of prompts
 - The two agree with GPT $\sim 75\text{-}85\%$ of the time
 - Same level of agreement as between humans

Easy Interviews → Perceive Lower-Ability Colleagues

50% ↓ mention high-ability colleagues, 2.5x-to-6x ↑ belief firm hires anyone

Accurate beliefs?

Covariates

Engaging?

RJP

CGQ

HC

	Cross-section sample		Within-worker sample	
Panel A: DV = 1(High-ability colleagues)				
1(Easy interview)	-0.020*** (0.001)		-0.035*** (0.003)	-0.025*** (0.006)
Fraction others' interviews easy		-0.022*** (0.002)		
Mean DV for not easy	0.037	0.037	0.054	0.054
Panel B: DV = 1(Would accept anybody)				
1(Easy interview)	0.061*** (0.001)		0.043*** (0.003)	0.028*** (0.005)
Fraction others' interviews easy		0.074*** (0.003)		
Mean DV for not easy	0.012	0.012	0.011	0.011
N	421,112	421,112	20,627	20,627
Worker FE				✓

Where we are

- ▶ Yesterday: supervised machine learning — flexible function fitting, with train/test discipline — and embeddings: text and images as vectors whose geometry encodes meaning.
- ▶ This morning: how ML predictions enter econometric designs.
- ▶ Today, in three moves:
 1. how a **language model** works — tokens, attention, the transformer;
 2. what turns a text completer into **ChatGPT** — instruction tuning, RLHF, reasoning;
 3. what economists can **do with the result** — RAG, agents, and a research application.

The recipe

Data (the whole internet) + compute (billions of parameters) + ML (transformer architecture) + **human preference alignment**.

Outline

How a Language Model Works

From GPT to ChatGPT: Instruction Fine-Tuning

RLHF and Preference Learning

Reasoning Models

Retrieval-Augmented Generation (RAG)

Vision-Language Models

Text Measurement with LLMs

Outline

How a Language Model Works

From GPT to ChatGPT: Instruction Fine-Tuning

RLHF and Preference Learning

Reasoning Models

Retrieval-Augmented Generation (RAG)

Vision-Language Models

Text Measurement with LLMs

Tokens: the atoms of a language model

- ▶ A language model does not read words. It reads **tokens**: entries in a fixed vocabulary of roughly 50k–200k character sequences.
- ▶ **Byte-pair encoding** (BPE) builds this vocabulary from the corpus: start from single characters, repeatedly merge the most frequent adjacent pair. Frequent words end up as one token; rare words split into pieces.

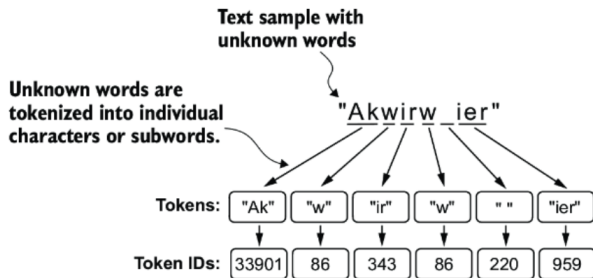


Figure 2.11 BPE tokenizers break down unknown words into subwords and individual characters. This way, a BPE tokenizer can parse any word and doesn't need to replace unknown words with special tokens, such as `<|unk|>`.

Language modeling = next-token prediction

- ▶ The task: given a corpus, learn $\Pr(w_{i+1} \mid w_{1:i})$ — a probability distribution over the *next* token given everything so far.
- ▶ Classic simplification is Markov assumption: condition only on last k tokens, $\Pr(w_{i+1} \mid w_{1:i}) \approx \Pr(w_{i+1} \mid w_{i-k:i})$. Modern LLMs have $k > 1$ million.
- ▶ The output is a **distribution** over the vocabulary of word pieces — these log-probabilities can be used as data.

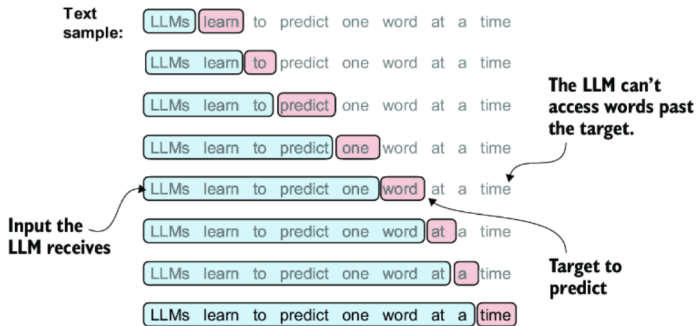


Figure 3.12 Given a text sample, extract input blocks as subsamples that

Perplexity: scoring a language model

- ▶ How well does a model predict a corpus of n tokens $\{w_1, \dots, w_n\}$?

$$\text{Perplexity} = 2^{-\frac{1}{n} \sum_{i=1}^n \log_2 \hat{\Pr}(w_i | w_{1:i-1})}$$

- ▶ Lower is better: a good model assigns high probability to the tokens that occur.
- ▶ Interpretation: the effective number of tokens the model is “choosing between” at each step.

From counting to function fitting

- ▶ The n-gram language model is just a ratio of corpus counts:

$$\hat{\text{Pr}}(w_{i+1} \mid w_{i-k:i}) = \frac{\#(w_{i-k:i+1})}{\#(w_{i-k:i})}$$

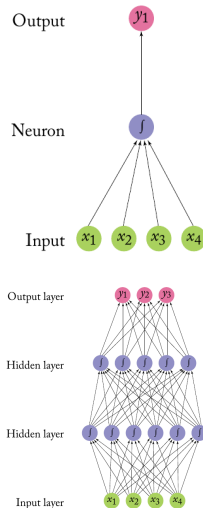
- ▶ It fails for the same reason a saturated regression fails: most cells are empty. Most word sequences never occur, so most estimates are zero or rest on a handful of observations.
- ▶ The **neural** language model replaces counting with smooth function fitting: embed the context tokens as vectors, feed them through a neural network, output a softmax over the vocabulary. Similar contexts now share statistical strength — exactly the embedding logic from yesterday.
- ▶ The network could be an MLP, a CNN, or an RNN. The architecture that won is the **transformer**.

Neural networks are stacked linear regressions

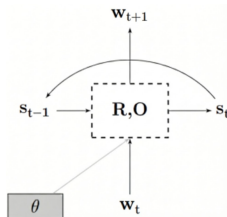
- ▶ A “neuron” is a linear regression with a squash: multiply each input by a learned coefficient, sum, and apply a nonlinear *activation function*:

$$\hat{y} = g(x \cdot \theta)$$

- ▶ A feed-forward network stacks neurons horizontally (many per layer) and vertically (many layers).
- ▶ In practice the activation is ReLU, $g(x \cdot \theta) = \max\{0, x \cdot \theta\}$, not the sigmoid.



Recurrent neural nets: the pre-transformer approach

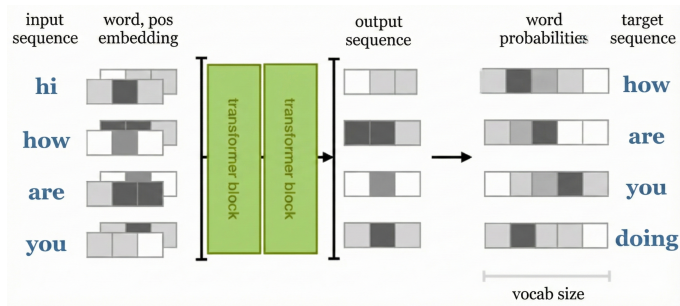


- ▶ At each token index t , a recursion function updates a state vector, $s_t = R(w_t, s_{t-1})$, and an output function predicts the next token, $\hat{w}_{t+1} = O(s_t)$.
- ▶ Key limitations:
 - ▶ sweeps through the document token by token — cannot be parallelized across time steps, so it learns slowly;
 - ▶ overly biased toward local context (nearby tokens);
 - ▶ training is brittle: the recursion is applied to its own output, so gradients explode or vanish.

Transformers overcome the limits of bag-of-words and RNNs

Since Vaswani et al. (2017), most deep learning for NLP uses the **transformer**

- ▶ built around a neural-net operation called **attention** (Bahdanau et al. 2014);
- ▶ intuitively, attention reads in entire document at once and learns the meaning of all words and all interactions between words, even at a distance
- ▶ the process is parallelizable during training — all tokens are processed simultaneously, unlike an RNN's sweep.



Attention: a learned soft lookup

- ▶ Think of a database of key–value pairs $D = \{(k_1, v_1), \dots, (k_m, v_m)\}$ and a query q . A dictionary lookup returns v_i where $k_i = q$: all-or-nothing weights.
- ▶ **Attention** generalizes the lookup to a weighted average:

$$\text{Attention}(q) = \sum_{i=1}^m a(q, k_i) v_i, \quad a(\cdot) \geq 0, \quad \sum_i a(q, k_i) = 1$$

- ▶ The weights come from a softmax over query–key similarity scores, so they are differentiable — the lookup itself can be *learned* by gradient descent.
- ▶ Linguistic reading: each word gets to query every other word in the document and pull in whatever is predictive.

Self-attention over word embeddings

- ▶ Take a sequence of tokens $\{w_1, \dots, w_{n_L}\}$ with embedding vectors $\{x_1, \dots, x_{n_L}\}$ of dimension n_E .
- ▶ A **self-attention layer** produces a new sequence of **context vectors** $h_1, \dots, h_{n_L}$:

$$h_i = \sum_{j=1}^{n_L} a(x_i, x_j) x_j, \quad a(x_i, x_j) = \text{softmax}\left(\frac{x_i \cdot x_j}{\sqrt{n_E}}\right)$$

- ▶ Each token's query is its own embedding; the keys and values are the other tokens' embeddings. Each h_i is a weighted average of the whole sentence, weighted by similarity to token i .
- ▶ This *basic* form has no parameters of its own — it works entirely through the learned embeddings. The production version (GPT, BERT) adds learned query/key/value projection matrices and multiple parallel **attention heads** per layer.

Why self-attention works

- ▶ Sentence: *the cat walks on the street*, with embeddings

$x_{\text{the}}, x_{\text{cat}}, x_{\text{walks}}, x_{\text{on}}, x_{\text{the}}, x_{\text{street}}$.

- ▶ Each output mixes the whole sentence:

$$h_{\text{cat}} = a(x_{\text{cat}}, x_{\text{the}}) x_{\text{the}} + a(x_{\text{cat}}, x_{\text{cat}}) x_{\text{cat}} + \dots + a(x_{\text{cat}}, x_{\text{street}}) x_{\text{street}}$$

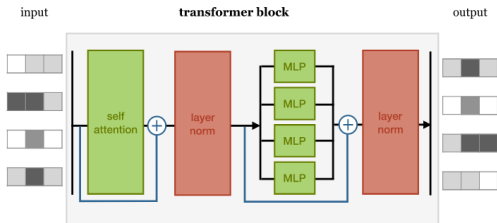
- ▶ Training shapes the embeddings so the dot products are *useful*: informative pairs (“cat”–“walks”) get weight, uninformative ones (“the”) get ignored.

Position embeddings: putting word order back

- ▶ Self-attention is a weighted average — it is **permutation invariant**. “Dog bites man” and “man bites dog” would encode identically.
- ▶ Fix: add a **position embedding** t_i (a learned vector for position index i) to each word embedding before the first attention layer:

$$h_{1:n_L}^0 = \{x_1 + t_1, \dots, x_{n_L} + t_{n_L}\}$$

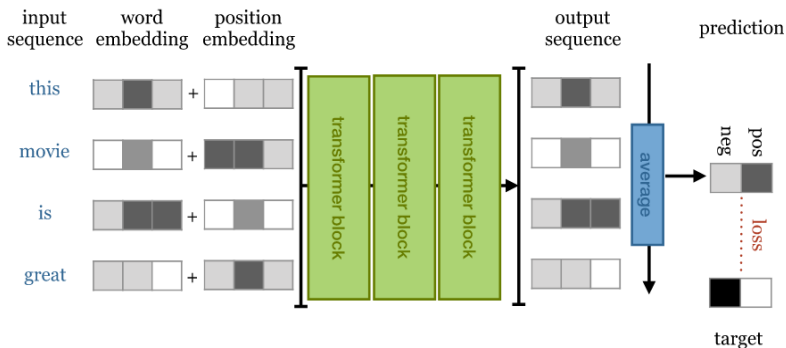
- ▶ Modern models instead *rotate* the embeddings by position (RoPE)



- ▶ One block = a **self-attention layer** followed by a small **MLP** at each position, plus two tricks that make deep stacks trainable: **residual connections** (the input is added back to each layer's output) and **layer normalization** (each vector rescaled to unit variance).

- ▶ Blocks chain: block l 's output $h_{1:n_L}^l$ is block $l + 1$'s input.
- ▶ The MLPs hold roughly **two thirds** of each block's parameters — remember this Thursday, when mixture-of-experts models multiply exactly that part.
- ▶ some say MLPs hold “knowledge”, while attention heads do “retrieval”

Stack blocks, add a head: a complete model



- Every transformer application is this picture with a different final layer: a classifier head (here), or a softmax over the vocabulary (a language model).

Three kinds of transformer language model

- ▶ **Autoregressive / decoder-only** (GPT): pre-trained to predict the next token, seeing only the past. Built for generation.
- ▶ **Autoencoding / encoder-only** (BERT): pre-trained to fill in a masked token, seeing the whole sequence in both directions. Built for representation — classification, embeddings.
- ▶ **Sequence-to-sequence** (Vaswani et al. 2017; T5, BART): an encoder reads, a decoder writes. Once standard for translation; now niche.

Masked ("causal") attention: how a decoder avoids cheating

- ▶ Training predicts every next token at once. **Masked attention** restricts each position to tokens at or before it:

$$\underbrace{h_i = \sum_{j=1}^{n_L} a(x_i, x_j) x_j}_{\text{encoder: sees all}} \longrightarrow \underbrace{h_i = \sum_{j=1}^i a(x_i, x_j) x_j}_{\text{decoder: past only}}$$

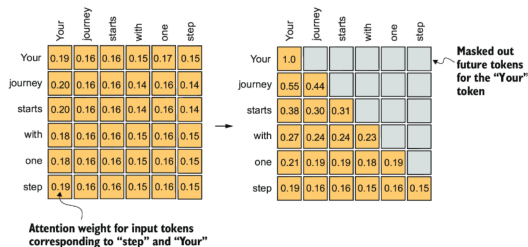
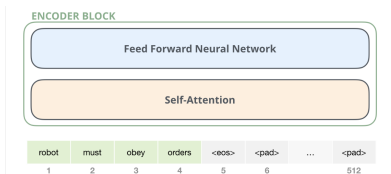
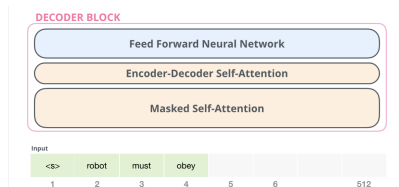


Figure 3.19 In causal attention, we mask out the attention weights above the diagonal such that for a given input, the LLM can't access future tokens when computing the context vectors using the attention weights. For example, for the word "journey" in the second row, we only keep the attention weights for

Encoder reads, decoder writes



An encoder block from the original transformer paper can take inputs up until a certain max sequence length (e.g. 512 tokens). It's okay if an input sequence is shorter than this limit, we can just pad the rest of the sequence.



Encoder block “reads”:

- ▶ observes the whole sequence, both directions;
- ▶ BERT: the classification and embedding workhorse.
- ▶ Interactive anatomy of a decoder:

<https://poloclub.github.io/transformer-explainer/>

Decoder block “writes”:

- ▶ masked attention: never sees future tokens;
- ▶ GPT — and every chat model in this lecture — is decoder-only.

GPT: scale up and pre-train

- ▶ **GPT = Generative Pre-trained Transformer**: a decoder-only stack trained on next-token prediction, then scaled.
 - ▶ **GPT-1** (2018): the Books corpus; 12 blocks, $\approx 117\text{M}$ parameters.
 - ▶ **GPT-2** (2019): 8M web documents (40 GB); 48 blocks, $\approx 1.5\text{B}$ parameters.
 - ▶ **GPT-3** (2020): Common Crawl + books + Wikipedia; 96 blocks, 12,288-dimensional embeddings, $\approx 175\text{B}$ parameters.
- ▶ Nothing conceptually new happens between GPT-1 and GPT-3 — the recipe is *the same objective on more data with more parameters*.
- ▶ What changed is what the models could *do*. That is where the story picks up.

Outline

How a Language Model Works

From GPT to ChatGPT: Instruction Fine-Tuning

RLHF and Preference Learning

Reasoning Models

Retrieval-Augmented Generation (RAG)

Vision-Language Models

Text Measurement with LLMs

Emergent zero-shot learning (GPT-1, GPT-2)

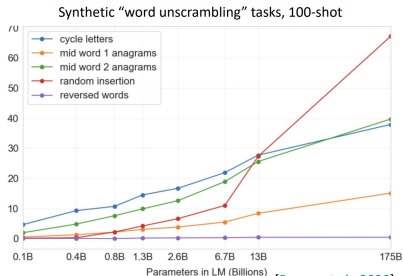
- ▶ Already with GPT (2018), and clearly with GPT-2 (“Language models are unsupervised multitask learners”, 2019), researchers observed **emergent zero-shot learning**.
- ▶ The model was never trained to answer questions or to summarize. But because the pre-training corpus contains such patterns, they can be elicited by formatting the prompt correctly:
 - ▶ Passage: [wikipedia excerpt for X] --- Q: Where was X born?
A: ...
 - ▶ [article] TL;DR: ...
- ▶ **Key idea:** the task specification is smuggled in through the text itself. There is no task-specific head and no labelled training set.

Main source for this section: Jesse Mu’s CS224n slides,

<https://web.stanford.edu/class/cs224n/slides/cs224n-2023-lecture11-prompting-rlhf.pdf>

Emergent few-shot learning (GPT-3)

- ▶ “Few-shot learning”: specify the task and give a handful of worked examples *inside the prompt*, before the query.
- ▶ Became strikingly effective with GPT-3 (“Language models are few-shot learners”, 2020). No gradient updates are involved — this is **in-context learning**.
- ▶ Crucially, the benefit *grows with model scale*.



Chain-of-thought prompting

- ▶ Asking the model to “think step by step” before answering substantially improves multi-step arithmetic and logical reasoning.
- ▶ The intermediate tokens act as a scratchpad: the model conditions its final answer on its own reasoning trace.
- ▶ This is a purely *prompting* intervention — a preview of reasoning models.

Chain-of-thought prompting

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Base language models are not aligned

- ▶ A pre-trained LM maximizes $\Pr(w_{t+1} \mid w_{1:t})$ on internet text. That is *not* the same objective as being helpful.
- ▶ Ask a base model a question and it may continue with more questions, restate the prompt, drift into an unrelated web page, or confidently fabricate.

Language modeling $\neq$ assisting users

PROMPT	<i>Explain the moon landing to a 6 year old in a few sentences.</i>
COMPLETION	GPT-3 Explain the theory of gravity to a 6 year old. Explain the theory of relativity to a 6 year old in a few sentences. Explain the big bang theory to a 6 year old. Explain evolution to a 6 year old.

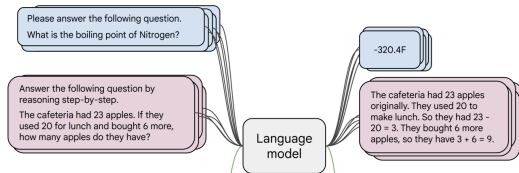
Language models are not *aligned* with user intent [[Ouyang et al., 2022](#)].

Instruction fine-tuning (IFT): continue training the pre-trained model, with the ordinary next-token objective, on a curated set of **(instruction, response)** pairs covering many tasks. Also called **supervised fine-tuning (SFT)**.

- ▶ Typically 10k–100k pairs — tiny, but reshapes behaviour dramatically.
- ▶ Payoff is **generalization across tasks**: train on diverse mixture of instructions and model then follows *held-out* instruction types it never saw.

Instruction finetuning

- **Collect examples** of (instruction, output) pairs across many tasks and finetune an LM



- Evaluate on **unseen tasks**

Q: Can Geoffrey Hinton have a conversation with George Washington?
Give the rationale before answering.

Geoffrey Hinton is a British-Canadian computer scientist born in 1947. George Washington died in 1799. Thus, they could not have had a conversation together. So the answer is "no".

What the instruction data looks like

Model input (Disambiguation QA)

Q: In the following sentences, explain the antecedent of the pronoun (which thing the pronoun refers to), or state that it is ambiguous.

Sentence: The reporter and the chef will discuss their favorite dishes.

Options:

- (A) They will discuss the reporter's favorite dishes
- (B) They will discuss the chef's favorite dishes
- (C) Ambiguous

A: Let's think step by step.

Before instruction finetuning

The reporter and the chef will discuss their favorite dishes.

The reporter and the chef will discuss the reporter's favorite dishes.

The reporter and the chef will discuss the chef's favorite dishes.

The reporter and the chef will discuss the reporter's and the chef's favorite dishes.

✗ (doesn't answer question)

- ▶ Prompts come from real API traffic plus labeller-written prompts; responses are written by trained human annotators.
- ▶ Diversity of *task type* matters more than raw volume.

Limitations of instruction fine-tuning

1. **Collecting task data is expensive.** Every example is a human-written demonstration.
2. **Many important tasks cannot be written as instructions with correct answers.** Story generation, advice, summarization at the right level of detail.
3. **Token-level losses penalize all mistakes equally.** A factual error and a stylistic quirk cost the same.
4. **Human-written answers may be suboptimal.** The model is forced to imitate one particular phrasing even when it could produce a better one.

Consequence

We need a training signal defined over *whole outputs* and expressing *relative* quality — not a token-by-token imitation loss.

Outline

How a Language Model Works

From GPT to ChatGPT: Instruction Fine-Tuning

RLHF and Preference Learning

Reasoning Models

Retrieval-Augmented Generation (RAG)

Vision-Language Models

Text Measurement with LLMs

Aligning language models with reinforcement learning

- ▶ Let $p_\theta(y \mid x)$ be a language model with parameters θ , giving the probability of output y given prompt x , with samples $\hat{y}(x)$.
- ▶ Suppose we have a reward (utility) function $U(y, x) \in \mathbb{R}$ — for example a human evaluation of the output.
 - ▶ $U(\cdot)$ need *not* be differentiable. And it can't really be differentiable for LLMs, because **utility comes from realized sequences, rather than token probabilities.**
- ▶ We want to choose θ to maximize expected reward over generated samples $\hat{y}_\theta(x) \sim p_\theta(y \mid x)$:

$$\theta^* = \arg \max_{\theta} \mathbb{E}\{U(\hat{y}_\theta(x), x)\}$$

- ▶ By gradient ascent:

$$\theta_{t+1} := \theta_t + \alpha \nabla_{\theta_t} \mathbb{E}\{U(\hat{y}_\theta(x), x)\}$$

- ▶ $\rightarrow$ solvable with **policy gradient** methods from reinforcement learning.

Policy gradients in three slides (1)

- For a sample of generations S ,

$$\nabla_{\theta} \mathbb{E}\{U(\hat{y}_{\theta}(x), x)\} = \sum_{y \in S} U(\hat{y}_{\theta}(x), x) \nabla_{\theta} p_{\theta}(y | x)$$

- The “log derivative trick”:

$$\begin{aligned} \nabla_{\theta} \log p_{\theta}(y | x) &= \frac{1}{p_{\theta}(y | x)} \nabla_{\theta} p_{\theta}(y | x), \\ \implies \nabla_{\theta} p_{\theta}(y | x) &= p_{\theta}(y | x) \nabla_{\theta} \log p_{\theta}(y | x). \end{aligned}$$

- Substituting:

$$\begin{aligned} \sum_{y \in S} U(\hat{y}_{\theta}(x), x) \nabla_{\theta} p_{\theta}(y | x) &= \sum_{y \in S} p_{\theta}(y | x) U(\hat{y}_{\theta}(x), x) \nabla_{\theta} \log p_{\theta}(y | x) \\ &= \mathbb{E}\{U(\hat{y}_{\theta}(x), x) \nabla_{\theta} \log p_{\theta}(y | x)\} \end{aligned}$$

Policy gradients in three slides (2)

- From the previous slide,

$$\underbrace{\nabla_{\theta} \mathbb{E}\{U(\hat{y}_{\theta}(x), x)\}}_{\text{left-hand side}} = \underbrace{\mathbb{E}\{U(y, x) \nabla_{\theta} \log p_{\theta}(y \mid x)\}}_{\text{right-hand side}}.$$

- The left-hand side cannot be computed directly because $U(\cdot)$ is not differentiable. Estimate the right-hand side by Monte Carlo, using $y_i \sim p_{\theta}(\cdot \mid x)$:

$$\mathbb{E}\{U(y, x) \nabla_{\theta} \log p_{\theta}(y \mid x)\} \approx \frac{1}{m} \sum_{i=1}^m U(y_i, x) \nabla_{\theta} \log p_{\theta}(y_i \mid x).$$

Policy gradients in three slides (3)

- ▶ Substituting the Monte Carlo estimate into gradient ascent gives

$$\theta_{t+1} := \theta_t + \alpha \frac{1}{m} \sum_{i=1}^m U(y_i, x) \nabla_{\theta} \log p_{\theta}(y_i | x)$$

- ▶ if $U(y_i, x)$ is positive, change θ to *increase* $p_{\theta}(y_i | x)$;
 - ▶ if $U(y_i, x)$ is negative, change θ to *decrease* it.
- ▶ Under regularity conditions this converges to θ^* .

How to model human preferences $U(\cdot)$

Problem 1: a human in the loop is far too expensive to evaluate $U(y, x)$ for every y, x encountered during training.

- Solution: train a **reward model** $U_\phi(y, x)$ — itself a fine-tuned LM with a scalar regression head — and use it instead of ground truth.

Problem 2: human preference scores are noisy and poorly calibrated across annotators.

- Solution: use **pairwise comparisons** — revealed preference! — rather than cardinal scores. Fit a Bradley–Terry model by minimizing

$$L(\phi) = -\frac{1}{|D|} \sum_D \log \sigma(U_\phi(y_0, x) - U_\phi(y_1, x))$$

where D is a dataset of pairs (y_0, y_1) and y_0 is the preferred output.

Interpretation for economists

U_ϕ is an estimated utility function of the representative annotator.

InstructGPT: where the preference data comes from

- ▶ Annotation team for InstructGPT: about **40 contractors** recruited through Upwork and ScaleAI.
- ▶ Screened with a qualification task covering flagging sensitive content, judging completion quality, writing demonstrations, and sensitivity to demographic group membership.
- ▶ Annotators see one prompt with several candidate completions, and rank them.

The cost of preferences

Nathan Lambert: “From the Llama 2 numbers and other rumors, it’s likely Meta spent \$10–20 million plus on preference data. This is just for the final model and may not include extensive experimentation and evaluation.”

- ▶ **Who these annotators are is an economic and political question**, not just an engineering detail.

RLHF: the full objective

Ingredients:

- ▶ a pre-trained (and instruction-tuned) model $p_{\theta}^{PT}(y | x)$, with θ now frozen;
- ▶ a reward model $U_{\phi}(y, x)$ estimated from pairwise comparisons.

Procedure:

- ▶ Make a trainable copy p_{ψ}^{RL} , initialized at p_{θ}^{PT} .
- ▶ Use RL on samples $(x, y) \sim p_{\psi}^{RL}$ to solve

$$\psi^* = \arg \max_{\psi} \sum_{y \in \mathcal{S}} \left(U_{\phi}(y, x) - \beta \log \frac{p_{\psi}^{RL}(y | x)}{p_{\theta}^{PT}(y | x)} \right)$$

- ▶ The second term is a **KL penalty** for diverging from the pre-trained model. Without it the policy collapses onto degenerate text that games the reward model.
- ▶ β is a hyperparameter (e.g. $\beta = 0.05$ in Stiennon et al. 2020).

Does it work? SFT + RLHF > SFT

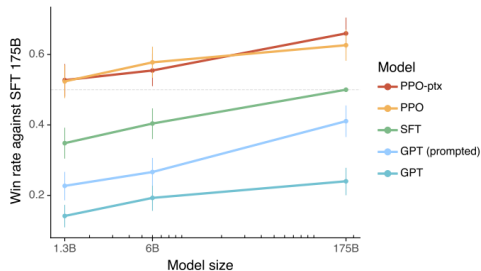


Figure 1: Human evaluations of various models on our API prompt distribution, evaluated by how often outputs from each model were preferred to those from the 175B SFT model. **Our InstructGPT models (PPO-ptx) as well as its variant trained without pretraining mix (PPO) significantly outperform the GPT-3 baselines (GPT, GPT prompted);** outputs from our 1.3B PPO-ptx model are preferred to those from the 175B GPT-3. Error bars throughout the paper are 95% confidence intervals.

- ▶ A 1.3B-parameter RLHF model is preferred to the 175B-parameter base GPT-3 (Ouyang et al. 2022) — alignment buys more than two orders of magnitude of scale on this metric.

Why RLHF, and not just more supervised data? (Goldberg 2023)

1. **Flexibility of wording.** Under imitation learning the model must reproduce the annotator's exact phrasing. But many wordings convey the same correct answer; SFT punishes the model for perfectly good alternatives it already knows how to produce.
2. **Positive and negative feedback.** SFT provides only positive examples; RLHF can say “not that”. In formal learning theory, without negative feedback an adversarial teacher can mislead the learner by withholding examples. *GPT-3 knows what to say; ChatGPT knows what **not** to say.*
3. **Preventing hallucination.** For knowledge-seeking queries the model must be able to *refuse*. SFT effectively teaches it to lie: since every demonstration contains an answer, the model learns always to produce one. RLHF can reward abstention.

Beyond PPO: DPO and GRPO

- ▶ **PPO** (proximal policy optimization) is on-policy: you must hold the policy, a frozen reference model, the reward model, and a value model in memory at once. Slow and fiddly.
- ▶ **DPO** (direct preference optimization): the KL-regularized objective has a closed-form optimal policy. Invert it and optimize preferences *directly* with a classification loss — no reward model, no sampling loop.
- ▶ **GRPO** (used in DeepSeek-R1): drop the learned value model. Sample a *group* of completions per prompt and use the group's mean reward as the baseline. Cheaper and more stable.

How smart can RLHF models get?

Andrej Karpathy:

- ▶ *“Instead of the mysticism of ‘asking an AI’, think of it more as ‘asking the average data labeler’ on the internet.”* With the caveat that in code, math, and creative writing the companies hire skilled labellers.
- ▶ *“RLHF moves the performance to ‘discriminative human’ grade, up from SFT which is at ‘generative human’ grade . . . because discrimination is easier for an average person than generation”* — label which of these five poems is best versus write a poem.
- ▶ *“You also get a separate boost from the wisdom of crowds effect . . . your LLM performance is not at human level, but at ensemble-of-human level.”*

The implied ceiling

With RLHF alone, the best you can hope for is a model whose answers a panel of top human experts would prefer over any alternative. **To go beyond human preferences you need a different reward signal** — which is where we go next.

Outline

How a Language Model Works

From GPT to ChatGPT: Instruction Fine-Tuning

RLHF and Preference Learning

Reasoning Models

Retrieval-Augmented Generation (RAG)

Vision-Language Models

Text Measurement with LLMs

What was still missing after GPT-4?

The “Sparks of AGI” paper (April 2023) catalogued failures that scale alone did not fix: arithmetic slips, planning errors, inability to backtrack, confident wrong answers.

- ▶ At least two things were missing:
 1. **information retrieval**, to access a fresh knowledge base;
 2. the ability to **take actions** and interact with an environment, rather than only writing text.
- ▶ And arguably a third:
 3. the ability to **reliably extrapolate and reason** through exploration and self-correction.

Item 3 is this section; items 1 and 2 are the next two sections.

How LLMs got good at math

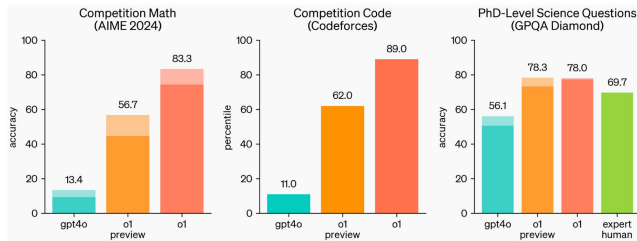
Category		Apply filter		Math			
Math		☐ Style Control		#models: 227 (99%) #votes: 384,871 (13%)			
		☐ Show Deprecated					
Rank* (UB)	Delta	Model	Arena Score	95% CI	Votes	Organization	License
1	0	Gemini-2.5-Pro-Exp-03-25	1417	+17/-20	1152	Google	Proprietary
1	1	o3-2025-04-16	1403	+39/-42	241	OpenAI	Proprietary
1	8	o4-mini-2025-04-16	1400	+43/-36	200	OpenAI	Proprietary
2	2	GPT-4.5-Preview	1377	+12/-17	1626	OpenAI	Proprietary
2	1	Gemini-2.5-Flash-Preview-04-17	1363	+20/-22	453	Google	Proprietary
2	7	DeepSeek-R1	1361	+13/-14	1601	DeepSeek	MIT
2	15	o3-mini-high	1359	+12/-13	2832	OpenAI	Proprietary
3	8	o1-2024-12-17	1354	+9/-9	3252	OpenAI	Proprietary

What is different about these models?

Scaling test-time compute

- ▶ Classic scaling: spend more compute at *training* time.
- ▶ New axis: spend more compute at *inference* time — let the model generate a long internal reasoning trace before answering.
- ▶ Accuracy improves roughly log-linearly in the number of thinking tokens, over several orders of magnitude.

Highlight of LLMs in 2024: the advancement of reasoning models



OpenAI o1 started to achieve impressive performance across various challenging reasoning tasks in math, coding, STEM, etc.

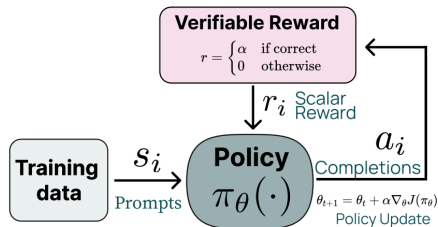
What “reasoning” models add

Relative to a standard RLHF chat model, a model like DeepSeek-R1 adds three ingredients:

1. **Long internal traces.** The model is allowed to “think” and “reflect”, generating many internally observed tokens before emitting an external answer.
2. **Reasoning SFT.** Additional supervised fine-tuning on a large set of worked examples containing step-by-step derivations, verifications, and self-corrections.
3. **RLVR — reinforcement learning from verifiable rewards.** The model solves math and code problems by guessing and verifying, and receives reward only when the answer is provably correct.

<https://www.interconnects.ai/p/deepseek-r1-recipe-for-o1>

RLVR: reinforcement learning from verifiable rewards



- ▶ Confront the model with thousands (even millions) of math and coding problems; let it guess, reflect, and verify until it finds a checkable solution. Reward only if correct.
- ▶ Gradually make the RLVR tasks harder and harder – model gets better and better at those tasks.

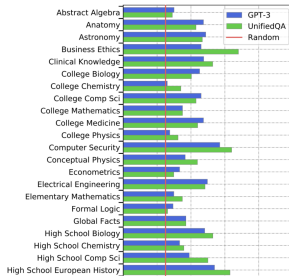
RLVR: What can it teach?

- ▶ With RLVR, reward is no longer a noisy estimate of average human preference.
- ▶ It is ground truth — so the human-preference ceiling does not bind. Also reduces hallucination by teaching self-critique.
- ▶ **Limitation:** it needs a verifier:
 - ▶ Easy for math and code
 - ▶ Hard for more subjective domains, e.g. writing, summarization, social-science.

Evaluating LLMs: knowledge benchmarks

Massive Multitask Language Understanding (MMLU) [\[Hendrycks et al., 2021\]](#)

New benchmarks for measuring LM performance on 57 diverse *knowledge intensive* tasks



- ▶ **MMLU**: about 16k multiple-choice questions across 57 subjects.
- ▶ **BIG-Bench**: a large heterogeneous suite scored with exact-match and string-similarity metrics.
- ▶ **GPQA**: 26,529 graduate-level, “Google-proof” questions across 285 fields.
- ▶ **Caution**: benchmarks leak into training corpora. Contamination makes published numbers hard to interpret.

Evaluating LLMs: LLM-as-judge

Arena-Hard-Auto (<https://arxiv.org/abs/2406.11939>):

- ▶ **Tasks:** 500 challenging real user queries (open-ended software engineering problems, math questions) plus 250 creative-writing queries sourced from Chatbot Arena.
- ▶ **Evaluation:** frontier LLMs act as judges, making *pairwise* comparisons between two models' answers — a cheap, fast approximation to human preference. The paper reports 98.6% correlation with human judgments.
- ▶ The judge writes its own answer first, then identifies errors in both candidates, then emits a verdict such as `[[A>B]]`.

Connection to applied econ

LLM-as-judge is now a standard measurement instrument in applied AI. Same lessons as human coders: report agreement with human labels on a validation sample, and worry about systematic bias (position effects, verbosity bias, self-preference).

LLM training through an economic lens

- ▶ **Pre-training** — learn the state space, and part of the action space.
- ▶ **Instruction fine-tuning (IFT)** — focus on the relevant parts of the action space.
- ▶ **Reinforcement learning with human feedback (RLHF)** — learn the user's preferences.
- ▶ **Reinforcement learning from verifiable rewards (RLVR)** — learn structured reasoning (Vafa et al. 2024).

The whole pipeline is a sequence of increasingly economic objects: a world model, a feasible action set, a utility function, and a solver.

Some interesting questions for economists

- ▶ How should human preference data be collected?
- ▶ How should the preference data be processed or weighted?
- ▶ What can economists say about the specification of the utility function?
 - ▶ e.g., should it be transitive?
- ▶ What are the implications for the model's worldview from RLHF vs. RLVR?
 - ▶ RLHF is a probabilistic approach — encourages interpolation.
 - ▶ RLVR is a discrete approach — discourages interpolation.

Outline

How a Language Model Works

From GPT to ChatGPT: Instruction Fine-Tuning

RLHF and Preference Learning

Reasoning Models

Retrieval-Augmented Generation (RAG)

Vision-Language Models

Text Measurement with LLMs

“Hallucination” in LLMs

- ▶ Ask a 2022-vintage ChatGPT “who is the US president today” and you will not get a good answer.
- ▶ Even with a 2025 model, niche questions — “who is the mayor of a small town in Graubünden?” — often produce confident, wrong answers.
- ▶ These wrong answers are called **hallucinations**: fluent, plausible, false.
- ▶ **Why they happen**: the fact was absent from pre-training, or appeared too rarely and too indirectly for the model to recall reliably from its internal world representation. The model has no mechanism for noticing the gap.

For empirical work

Hallucination is a *measurement error* problem, and it is not classical: it is correlated with obscurity, which is often correlated with the variables we care about — small places, minority groups, older documents.

Two solutions that do not work

1. **Re-train the model every day.** Infeasible on cost, and it would not help with facts that are rare rather than recent.
2. **Put all the facts in the system prompt.** Context windows are large but not that large, and cost scales with context length.

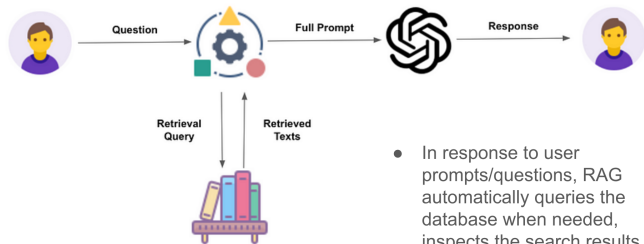
The workable answer

Give the model *selective* access to an external, trustworthy knowledge base at query time — **retrieval-augmented generation**.

Addressing hallucination with RAG

- ▶ Retrieval-augmented generation gives the model access to:
 - ▶ a database of useful and reliable knowledge, and
 - ▶ a search engine for retrieving from it efficiently.
- ▶ Retrieved passages are inserted into the prompt, and the model answers *conditional on the evidence*.
- ▶ Bonus: RAG returns **citations** to source documents for checking.

RAG = Retrieval Augmented Generation



- In response to user prompts/questions, RAG automatically queries the database when needed, inspects the search results, and uses them to respond.

How to index the documents

- ▶ **Sparse / lexical: BM25.** Rank documents by a TF-IDF-like similarity to the query, using an inverted index over words to discard irrelevant documents fast. Very fast, but requires exact word overlap.
- ▶ **Dense / embedding-based.** Encode chunks and queries with a sentence-embedding model (e.g. MPNet from SentenceTransformers), and retrieve by cosine similarity.
 - ▶ Handles paraphrase and synonymy.
 - ▶ Documents must be segmented into **overlapping chunks** to fit the encoder's context.
 - ▶ Approximate nearest-neighbour search (e.g. FAISS, Johnson et al. 2017) makes this fast at the scale of tens of millions of chunks.
- ▶ **Hybrid** is standard practice: BM25 for a cheap first pass, then embeddings or a cross-encoder (relevance classifier) to re-rank the top results.

RAG is powerful but fails too often → boost context

- ▶ **Inference is easier than retrieval.** Modern LLMs are so good at reading a provided document that almost all open-QA errors come from the *retriever*. Once the correct evidence is in context, the answer follows. → high returns to “context engineering”.
- ▶ Context windows have exploded:
 - ▶ GPT-3.5 (2022): 4K tokens ($\approx$ 6 pages)
 - ▶ GPT-4 (2023): 8K tokens ($\approx$ 12 pages)
 - ▶ Claude Sonnet 4 (2025): 200K tokens ($\approx$ 700 pages)
 - ▶ Gemini 2.5 (2025): 1M tokens ($\approx$ 3,000 pages)
- ▶ When context is abundant, **search becomes navigation**. Instead of retrieving fragments, load full documents, follow references, and build comprehensive understanding.

Agentic search vs. RAG (Bustamante 2025)

- ▶ Traditional RAG is a long pipeline: chunk → embed → retrieve → rerank → generate. Each step can fail, and errors cascade.
- ▶ **Agentic search:** give the model simple tools (grep, file navigation, a shell) and let it investigate iteratively — reading full documents, following cross-references, and accumulating understanding.

Illustration: “What is the company’s litigation exposure?”

- ▶ RAG returns chunks mentioning “litigation” → finds \$500M.
- ▶ An agent follows cross-references across sections and footnotes → finds \$5.1B, ten times more.

“The future belongs to agents that navigate and reason, not pipelines that retrieve fragments.”

<https://www.nicolasbustamante.com/p/the-rag-obituary-killed-by-agents>

Outline

How a Language Model Works

From GPT to ChatGPT: Instruction Fine-Tuning

RLHF and Preference Learning

Reasoning Models

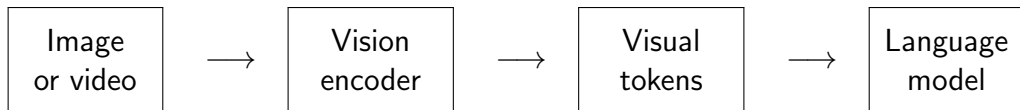
Retrieval-Augmented Generation (RAG)

Vision-Language Models

Text Measurement with LLMs

From multimodal embeddings to generative VLMs

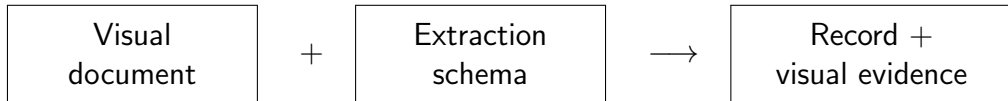
- ▶ Yesterday we looked at text-image embeddings like CLIP.
- ▶ A **generative vision-language model (VLM)** goes one step further: it conditions a language model on visual input and generates a response.



- ▶ An adapter or cross-attention layer maps the vision encoder's output into the language model's representation space.
- ▶ **Multimodal IFT/RLHF/RLVR** teaches the model to answer questions about images, follow visual instructions, and produce structured output from them.

VLMs can read documents as images

- ▶ VLMs can jointly interpret language, layout, and graphics: charts, forms, receipts, maps, newspapers, and scanned archival records.
 - ▶ e.g. in "Read the paper, write the code", we had a VLM (GPT-5-mini) extract the full content of regression tables in JSON.
- ▶ Ask for a structured record, but retain the source page, crop, quoted evidence, and model response so every extracted value can be audited.



Outline

How a Language Model Works

From GPT to ChatGPT: Instruction Fine-Tuning

RLHF and Preference Learning

Reasoning Models

Retrieval-Augmented Generation (RAG)

Vision-Language Models

Text Measurement with LLMs

What LLMs can do for text analysis

- ▶ In the realm of text data, LLMs work like very cheap and fast research assistants.
- ▶ Whatever you could trust a good undergrad RA to do, LLMs should be able to do.
- ▶ We will illustrate with a project that measures the **cultural content of historical proverbs** — a task that was impossible at scale before aligned LLMs.

Labeling documents with LLMs

- ▶ For a labeling task where humans get high agreement, LLMs provide very good performance.
 - ▶ Performance is comparable to attentive humans since roughly GPT-4o (May 2024).
 - ▶ Use the API and annotate documents programmatically from Python or R — highly parallelizable.
 - ▶ When available, set temperature = 0 so outputs are (almost) reproducible.
- ▶ For a labeling task where humans do *not* agree, LLMs are not a magical solution.
 - ▶ If the task is too difficult for your labelers, get better labelers.
 - ▶ If LLM outputs cannot be externally validated, they cannot be relied on.

How to write prompts

1. Use your best knowledge and intuition.
2. LLMs are very good at following long, detailed instructions, as long as they are coherent. Be specific.
3. Ask an LLM for advice on improving a prompt.
4. Try the prompt on a small sample; find errors and edge cases; update.
5. Prepare a longer human validation set; find further edge cases.
6. Use Co-DETECT (<https://github.com/EdisonNi-hku/Co-DETECT/>) to systematically find and address edge cases (Xiong et al. 2025).

What are the “best” LLMs for economics research?

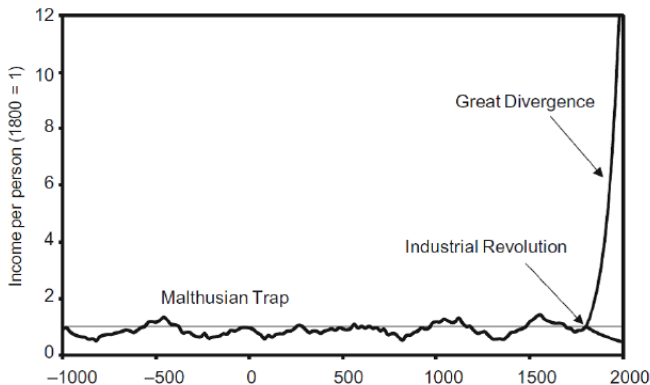
- ▶ For text analysis: Yang et al. (2025) compare 15 LLMs on annotation tasks from 14 political-science papers and all get very high accuracy ($>99\%$).
- ▶ For text annotation of low-dimensional unstructured outputs:
 - ▶ the frontier models have been excellent for about two years — differences between them are small relative to differences in your prompt and validation;
 - ▶ the major providers (OpenAI, Anthropic, Google, xAI) are comparable at any point in time;
 - ▶ open-weights models (Qwen, Llama, DeepSeek) are excellent alternatives when you need reproducibility or data governance.
- ▶ For extracting high-dimensional structured outputs:
 - ▶ you will need a strong model, e.g. GPT 5.6 terra or above
 - ▶ the best open-weights models (eg recent large Qwen, Kimi, and GLM models) also work well for the complex tasks.

British Industrialization and Cultural Change: Evidence from the Usage of Proverbs

Elliott Ash, ETH Zurich and CEPR
Melanie Meng Xue, LSE and CEPR

December 2025

The Industrial Revolution



- “The most fundamental transformation of human life in the history of the world recorded in written documents.” (Hobsbawm, 1969)

Proverbs and culture

**“Hold on, wait for the grasshoppers.”
“Many get into a dispute well that cannot get out well.”**

- ▶ Proverbs tend to be metaphorical and cryptic, with unusual syntactic structure:
 - ▶ that makes them easy to memorize and to transmit across generations, resistant to information loss or manipulation (Villers, 2022);
 - ▶ it also makes them hard to understand just from reading them — you need external knowledge.

New data on historical British proverbs

- ▶ **Lists of proverbs:** 26,000 collected from 5 historical sources — OCR'd the PDFs, regex extraction, post-processing, checking, cleaning.
- ▶ **Pre-industrial usage:** EEBO (Early English Books Online) — 17 million pages from 146,000 books, 1475–1750.
- ▶ **Post-industrial usage:** British Newspaper Archive — 40 million pages from more than 1,000 newspaper outlets, 1750–1900.

Explaining proverbs with AI

- ▶ Key challenge: we need not just the proverbs but **explanations** that consistently lay out their social implications.
 - ▶ Would require thousands of hours of work by human expert annotators.
 - ▶ Standard NLP tools do not work with metaphor or implicit meaning.

What LLMs can do that makes this project possible

- ▶ draw on a giant multilingual text knowledge base;
- ▶ interpret metaphorical language;
- ▶ infer and interpolate meaning.

What can we do now that we could not do before?

- ▶ **20 years ago:** annotate proverb explanations along some dimension, machine-classify the rest; learn LDA topics on the explanations.
 - ▶ **But how do we get the explanations?**
- ▶ **7 years ago:** use BERT for more accurate classification from labels; BERTopic for better, context-sensitive topics than LDA.
 - ▶ **But again: how do we get the explanations?**
- ▶ **3 years ago (RLHF models): use an aligned LLM to translate each proverb into an explanation of its cultural meaning** — then apply the familiar tools (BERT, topic models) to classify and interpret the explanations.

What about the future?

- ▶ **Today (RLHF LLMs):** LLM translates proverb → explanation; classifiers and topic models interpret the explanations.
- ▶ **Also today, but experimental (RLVR LLM agents):**
 - ▶ “search the web and give me a list of all English-language proverbs emphasizing the value of patience”;
 - ▶ “count the usage of these proverbs in a provided corpus by county and over time”.
- ▶ **What LLMs cannot do, and won’t be able to do:**
 - ▶ “count the usage of these proverbs by county and over time, but the corpus is not online”;
 - ▶ “give me a measure of the counties in England over time by their cultural valuing of patience”;
 - ▶ “what is the causal effect of industrialization on the valuing of patience?”

The prompt (GPT-4o)

You are a helpful research assistant with knowledge of the literary history of proverbs. Explain the meaning of this proverb in a few sentences of plain language. Interpret the proverb in terms of the implied beliefs, values, attitudes, norms, or ideology.

Structure your answer as follows: "This proverb means that... If a person uses this proverb, it implies that this person ... If this proverb is prevalent in a society, it implies that the society...".

Prompting matters (1): two versions of the ask

Old prompt:

Explain the meaning of this proverb in a few sentences of plain language, including an assessment of its moral, behavioral, or ethical implications. Begin your answer with: "This proverb means that...".

New prompt:

Explain the meaning of this proverb in a few sentences of plain language. Interpret the proverb in terms of the implied beliefs, values, attitudes, norms, or ideology. Structure your answer as follows: "This proverb means that... If a person uses this proverb, it implies that this person ... If this proverb is prevalent in a society, it implies that the society...".

Prompting matters (2): what changes in the output

Proverb: What belongs to the master is forbidden to the slave.

Old explanation: This proverb means that there are certain privileges or possessions that are reserved for those in positions of power . . . **The moral or ethical implication of this proverb is a critique of social injustice and the unfair distribution of resources and rights.**

New explanation: This proverb means that what is owned or controlled by someone in a position of power is not accessible to those in a subordinate position. If a person uses this proverb, it implies that this person recognizes and accepts the strict boundaries and inequalities between social classes. **If this proverb is prevalent in a society, it implies that the society has a clear hierarchy and reinforces the idea that those in power have exclusive rights over their possessions and privileges.**

- ▶ The old prompt made the LLM *moralize*; the new prompt makes it *describe* — the difference between an editorial and a measurement.

Clustering the explanations with BERTopic

(LLMs can't really learn topics inductively — clustering still does that job)

A. Patience



B. Managing Conflict



Hold on, wait for the grasshoppers.

Haste comes late in the end.

He who pours water hastily into a bottle spills more than goes in.

The Roman conquered by delay.

Haste makes waste and waste makes want, and want makes strife between the good man and his wife.

Nothing is well ordered that is hasty or precipitate.

He tires betimes that spurs too fast betimes, with eager feeding food will choke the feeder.

Many get into a dispute well that cannot get out well.

It is better to keep out of a quarrel than to make it up afterwards.

Dissensions like small streams at first begun, scarce seen they rise but gather as they run.

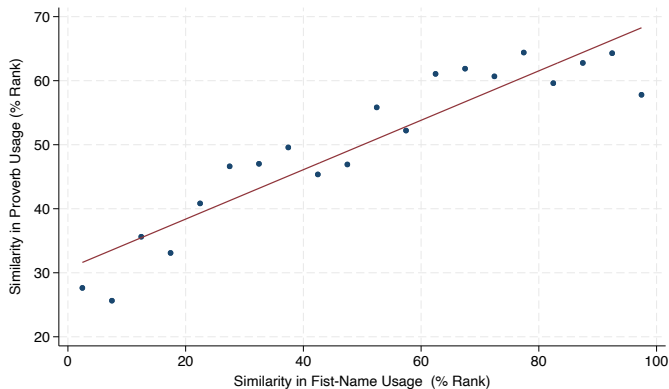
It is madness for a sheep to treat of peace with a wolf.

Wranglers never want words though they may matter.

Take things always by the smooth handle.

Happy is the man who keeps out of strife.

Proverb usage captures cultural distance



County-to-county cosine similarity in proverb topic frequencies, 1800–1900, against cosine similarity in first-name frequencies from the 1851 census. Spearman correlation = 0.26.

LLM Prompt: Work Ethic (Thompson 1967)

You are a helpful research assistant with knowledge of the literary and social history of proverbs. I will provide you with a proverb and its meaning.

Does the proverb emphasize the importance of work ethic, that is, a belief that work and diligence have a moral benefit and an inherent ability, virtue or value to strengthen character and individual abilities?

Read the proverb and its associated meaning and classify it into one of the following categories:

- "Definitely Yes": clearly, directly and strongly emphasizes the importance of work ethic.
- "Probably Yes": suggests the importance of work ethic, though less directly and strongly than "Definitely Yes".
- "Probably Not": suggests that work ethic is not important, though less directly and strongly than "Definitely Not".
- "Definitely Not": clearly, directly and strongly contradicts or dismisses the importance of work ethic.
- "Unrelated": unrelated to work ethic.

Make sure that do not use any exterior or prior knowledge you have when analyzing and do not infer meaning beyond what is explicitly stated. Base your answers only on the proverb provided.

Please also provide a short one-sentence explanation for your classification.

- **Work Ethic (Thompson 1967)**

- ① First deserve, and then desire.
- ② Do not go to sleep early and rise late.
- ③ As is the garden such is the gardener.
- ④ Things hardly attained are longer retained .

Validations

- ▶ Human comparisons of our explanations to dictionaries.
- ▶ Human validation of the annotations of cultural dimensions.
- ▶ Validation that explanations and annotations make sense in context when used in newspapers, books, and web sites.
- ▶ Comparisons across other LLMs and other prompts.
- ▶ Intruder detection on the clusters.

The measurement lesson

An LLM is a coder, not an oracle: validate its output the way you would validate any human annotator.

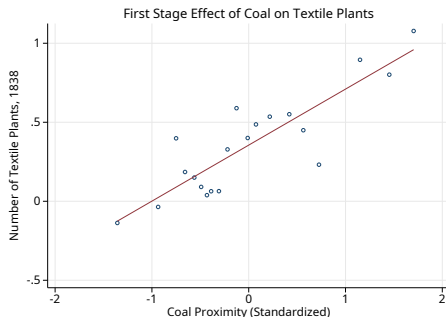
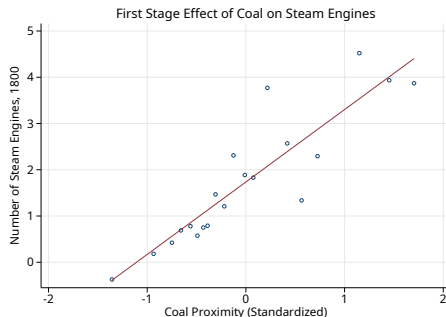
Cultural change in the Industrial Revolution

- ▶ Industrialization transformed England economically and culturally from the late 1700s, with labor shifting out of agriculture into manufacturing and services (Broadberry et al., 2015; Wallis et al., 2018).
- ▶ Thompson (1964): anecdotal evidence on the rise of a “working class” identity, and especially changes in attitudes around *time*.

The design

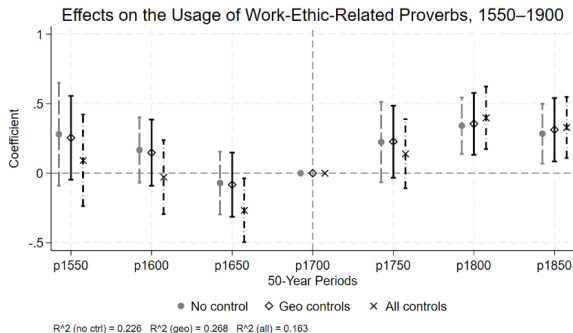
Assess how local proverb usage changed in localities experiencing industrialization, using **presence of coal reserves** as an exogenous shifter to earlier/more industrialization.

First-stage regression results



Steam engines in 1800 (left) and textile plants in 1838 (right) by county, against the share of parishes with coal. Both significant at $p < .01$; Kleibergen–Paap F-stat > 50 .

EFFECT OF INDUSTRIALIZING ON VALUING OF WORK ETHIC



Notes. Regression of usage of proverbs valuing work ethic by 50-year period on coal proximity for that county, interacted with period indicators. Regression include county and period FE's. 90% confidence intervals constructed with standard errors clustered by county.

How will you use LLMs for economics research?